



**Università
di Genova**

High Performance Computing Report Mandelbrot Algorithm

Andrea Cattarinich 5137057

November 28, 2025

Contents

1	Architectures	2
1.1	Personal Workstation	2
1.2	University Workstation	2
1.3	Google Colab	3
2	Mandelbrot Set	4
2.1	Definition	4
2.1.1	Point that belongs to the Mandelbrot set	4
2.1.2	Point that does NOT belong to the Mandelbrot set	4
2.2	Images	4
3	Hotspot Analysis	5
3.1	Optimization Problems	5
3.1.1	Solution and Results	5
3.2	Hotspot Identification	6
3.3	Conclusions	7
4	Conclusions	8

1 Architectures

For this project I have used three workstation, my personal one, the workstation provided by the University and Google Colab.

1.1 Personal Workstation

HP Envy 17-ae103nl

CPU

Model	Intel Core i7-6500U @ 2.50 GHz
Architecture	x86_64
Physical cores	2
Logical threads	4 (Hyper-Threading)

GPU

Model	NVIDIA GeForce GTX 950M
CUDA Compute Capability	5.0
Driver version	572.16
CUDA Version	12.8

Software and Setup

Visual Studio	Visual Studio Community 2022 (C++ Desktop workload)
CUDA Toolkit	Installed manually
WSL2	Enabled
NVIDIA Drivers	Version 572.16 (for WSL2)

1.2 University Workstation

Room 210

CPU

Model	Intel Core i9-12900K (12th Gen)
Architecture	x86_64
CPU(s)	24
Threads per core	2
Cores per socket	16
Socket(s)	1
Max frequency	4.02 GHz
Profiling tools	Intel VTune Profiler, Intel Advisor

To use the Intel oneAPI tools, the environment was loaded with:

```
source /opt/intel/oneapi/setvars.sh
```

1.3 Google Colab

CPU

Model	Intel(R) Xeon(R) CPU @ 2.20GHz
Architecture	x86_64
Physical cores	1
Logical threads	2 (Hyper-Threading)
Socket(s)	1

GPU

Model	NVIDIA Tesla T4
CUDA Compute Capability	7.5
Driver version	550.54.15
CUDA Version	12.8

2 Mandelbrot Set

2.1 Definition

The Mandelbrot fractal is defined as a set of points in the complex plane. Each point corresponds to a complex number c .

For every c , we consider the iterative sequence:

$$z_0 = 0, \quad z_{n+1} = z_n^2 + c$$

We then observe the behavior of z_n as $n \rightarrow \infty$.

2.1.1 Point that belongs to the Mandelbrot set

Take $c = -1$. Then the sequence is

$$z_0 = 0, \quad z_1 = 0^2 + (-1) = -1, \quad z_2 = (-1)^2 + (-1) = 0, \quad z_3 = 0^2 + (-1) = -1, \dots$$

The results remain bounded, so $c = -1$ belongs to the Mandelbrot set.

2.1.2 Point that does NOT belong to the Mandelbrot set

Take $c = 1$. Then the sequence is

$$z_0 = 0, \quad z_1 = 0^2 + 1 = 1, \quad z_2 = 1^2 + 1 = 2, \quad z_3 = 2^2 + 1 = 5, \dots$$

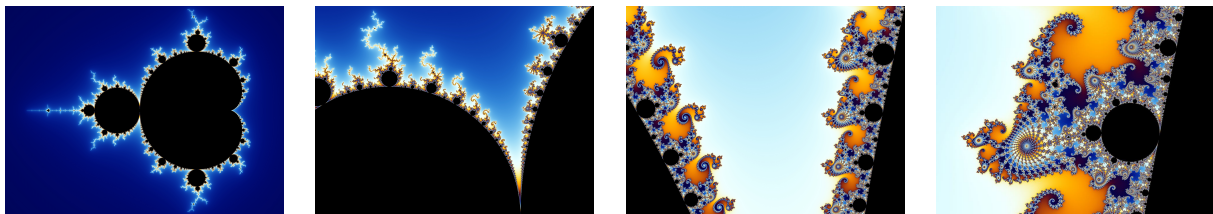
The values grow up to infinite, so $c = 1$ does not belong to the Mandelbrot set.

In our specific case, the escape condition is defined as:

$$|z_i| > 2$$

2.2 Images

This process gives rise to the famously intricate structure of the Mandelbrot fractal. Here a simple explanation [1] or here a more complex one [2] and here an online demo [3].



The color represents how quickly the values of z_n grow:

Black - the sequence remains bounded;

Other colors - the sequence escapes to infinity, with the color indicating how fast it diverges.

3 Hotspot Analysis

3.1 Optimization Problems

Questa istruzione non e' vettorizzabile con MSVC, e per tanto, l'esecuzione risulta essere poco performante.

Mentre con Linux G++, l'operazione sembra ottimizzare `std::complex`.

```
1 z = pow(z, 2) + c;
```

L'operazione `std::complex` utilizza l'overloading, in particolare `operator+` e `operator*`, creando un overhead e un conseguente carico sulla CPU. Inoltre, l'operazione `abs(z)` calcola la radice quadrata ogni volta.

Per rimuovere questa dipendenza dovuta dai numeri complessi, ho realizzato un miglioramento nel codice che mi consente di usare solamente numeri reali. Infatti, sfruttando la seguente relazione

$$z^2 = (a, b)^2 = (a^2 - b^2, 2ab)$$

Ho implementato:

$$z^2 = (\text{Re}(z), \text{Im}(z))^2 = (\text{Re}(z)^2 - \text{Im}(z)^2, 2 \text{Re}(z) \text{Im}(z))$$

Come segue:

```
1 double zRe = 0, zIm = 0;
2 for (int i = 1; i <= ITERATIONS; i++)
3 {
4     double oldRe = zRe;
5     double zRe2 = zRe*zRe; // Re(z)^2
6     double zIm2 = zIm*zIm; // Im(z)^2
7
8     zRe = (zRe2 - zIm2) + cRe;
9     zIm = 2*oldRe*zIm + cIm;
10
11     if(zRe2 + zIm2 >= 4)
12     {
13         image[pos] = i;
14         break;
15     }
16 }
```

3.1.1 Solution and Results

I risultati, come ci aspettavamo sono più performanti.

MSVC

```
cl /O2 /arch:AVX2 /fp:fast /EHsc src\*.cpp /Fe:build\*.exe
```

WSL

```
g++ -O2 -o build/* src/*.cpp
```

	mandelbrot	opt1	opt2
WSL	96s	30s (x3.2)	36s (x2.7)
Windows	-	40s	36 s

Utilizzeremo questo codice come baseline per lo sviluppo della restante parte del progetto.

3.2 Hotspot Identification

The measurements were performed using Intel VTune Profiler and the analysis was performed using Intel Advisor.

Function Stack	Total	Percentage (%)
Total	69.97	100.00
_start	69.97	100.00
__libc_start_main_impl	69.97	100.00
– main	69.97	100.00
— std::abs<double>	41.2121	58.90
— std::operator+<double>	13.6361	19.49
— std::pow<double>	10.712	15.31
— std::complex<double>::complex	0.0199997	0.03

The two main hotspots are the following instructions which takes the most fraction of the program execution.

Line 48

```
1 z = pow(z, 2) + c;
```

Line 51

```
1 if (abs(z) >= 2)
```

Source Line ▲	Source	CPU Time: Total	CPU Time: Self
31			
32	const auto start = chrono::steady_clock::now();		
33	for (int pos = 0; pos < HEIGHT * WIDTH; pos++)	0.0%	0.008s
34	{		
35	image[pos] = 0;	0.0%	0.012s
36			
37	const int row = pos / WIDTH;	0.0%	0.008s
38	const int col = pos % WIDTH;	0.0%	0.012s
39	const complex<double> c(col * STEP + MIN_X, row * STEP + MIN_Y);	0.0%	0.012s
40			
41	// z = z^2 + c		
42	complex<double> z(0, 0);		
43	for (int i = 1; i <= ITERATIONS; i++)		
44	{		
45	z = pow(z, 2) + c;	39.9%	3.570s
46			
47	// If it is convergent		
48	if (abs(z) >= 2)	59.9%	0.732s
49	{		
50	image[pos] = i;		
51	break;		
52	}		
53	}	0.0%	0.024s
54	}	0.0%	0.012s

3.3 Conclusions

The main hotspot is the DFT() function and its inverse IDFT(), which together account for over 99,05% of the total execution time. All other functions have negligible impact. This indicates that **parallelization and vectorization efforts should focus on the double loop inside DFT()**.

The main hotspot is the loop contained in lines 72-80 (source: src/omp_homework.c). The dominant operations contributing to the algorithm's computational intensity are the `sin()` and `cos()` trigonometric functions, which account for the majority of the execution time, as shown in the following code.

Source	Assembly				
Source Line ▲	Source				🔥 CPU Time: Total ⓘ CPU Time: Self ⓘ
31					
32	const auto start = chrono::steady_clock::now();				
33	for (int pos = 0; pos < HEIGHT * WIDTH; pos++)				0.0%
34	{				0.008s
35	image[pos] = 0;				0.012s
36					
37	const int row = pos / WIDTH;				0.0%
38	const int col = pos % WIDTH;				0.008s
39	const complex<double> c(col * STEP + MIN_X, row * STEP + MIN_Y);				0.012s
40					
41	// z = z^2 + c				
42	complex<double> z(0, 0);				
43	for (int i = 1; i <= ITERATIONS; i++)				
44	{				
45	z = pow(z, 2) + c;				39.9%
46					3.570s
47	// If it is convergent				
48	if (abs(z) >= 2)				59.9%
49	{				0.732s
50	image[pos] = i;				
51	break;				
52	}				
53	}				0.0%
54	}				0.024s
					0.012s

4 Conclusions

References

- [1] Simple Explanation of the Mandelbrot Set.
<https://www.wikihow.com/Plot-the-Mandelbrot-Set-By-Hand>
- [2] Mandelbrot Set – Formal Explanation.
https://en.wikipedia.org/wiki/Mandelbrot_set
- [3] Online Mandelbrot Demo.
<https://math.hws.edu/eck/js/mandelbrot/MB.html>